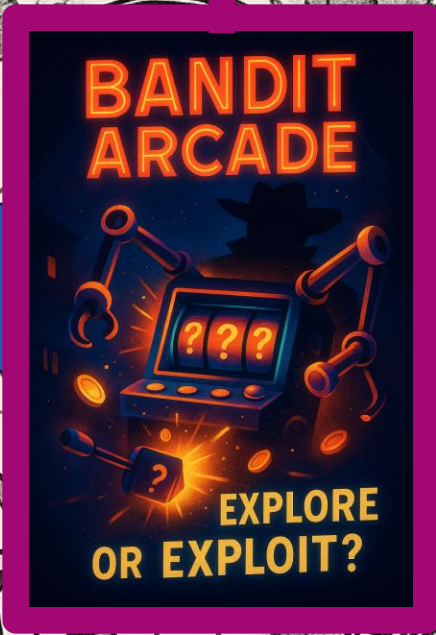
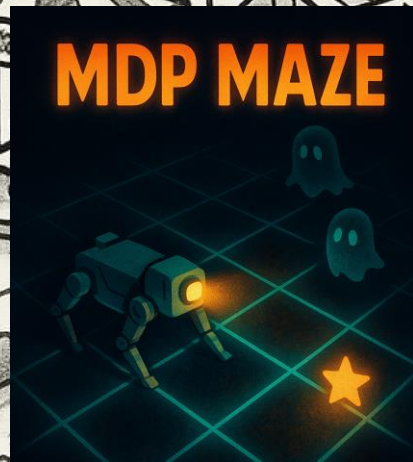


Reinforcement learning

AI and IT in Industry
2025



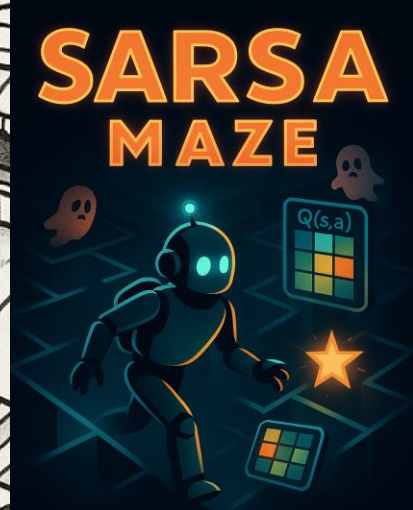


How do we find the optimal strategy?

Find the action that yields the highest cumulative reward!

How do we compute the cumulative reward?

Use the Bellman equations!

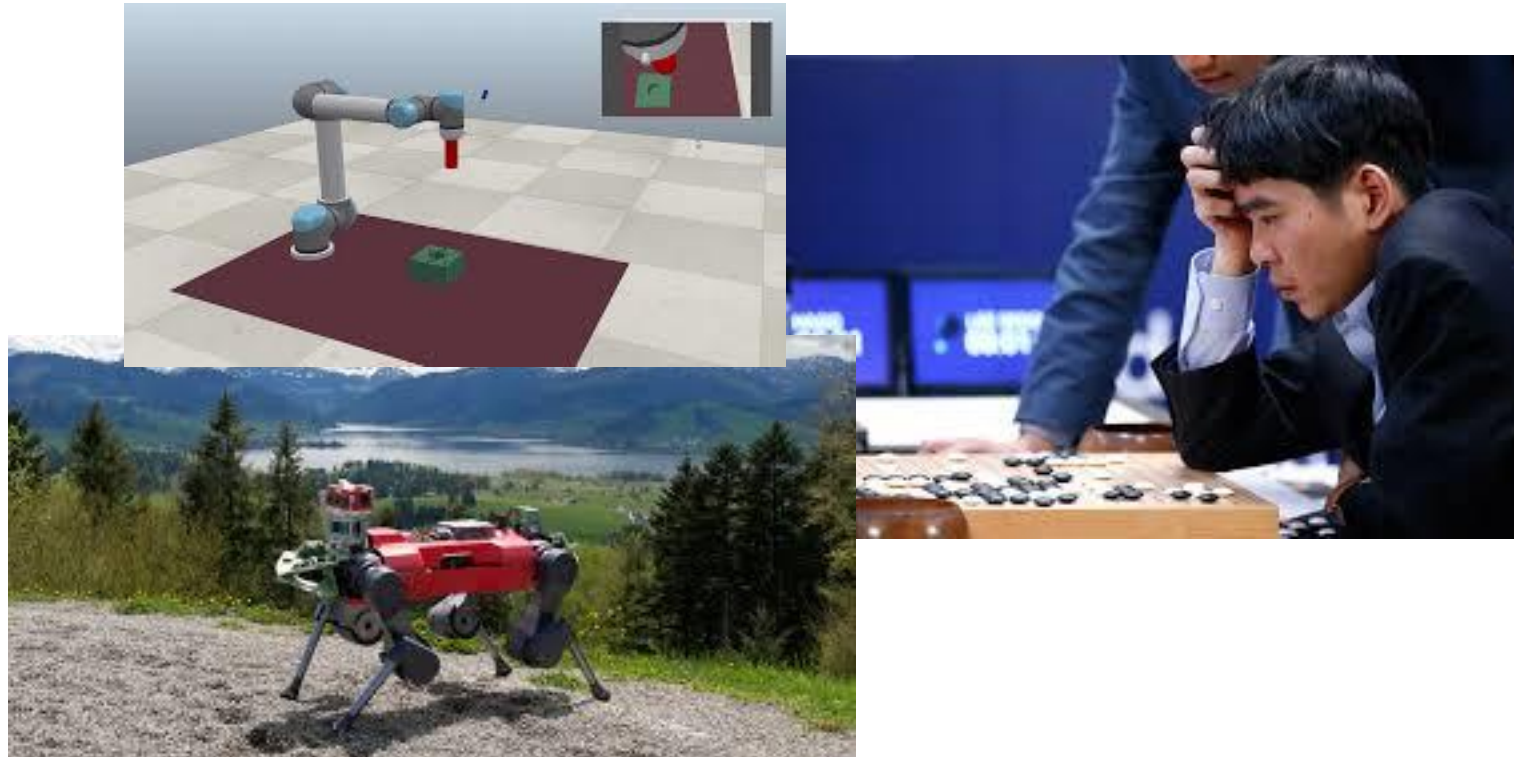


How do we compute the Bellman equations?

Use Robbins-Monro

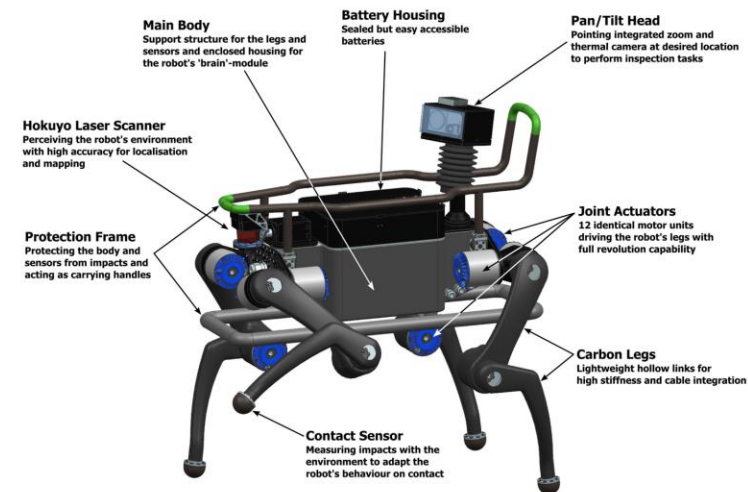
What is reinforcement learning (RL)?

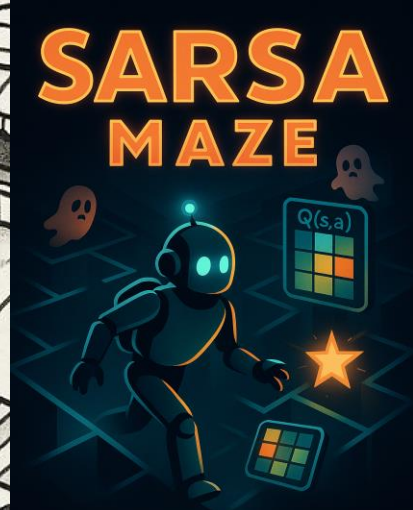
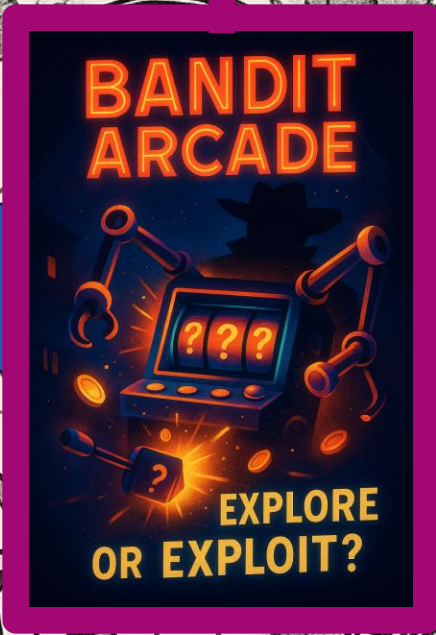
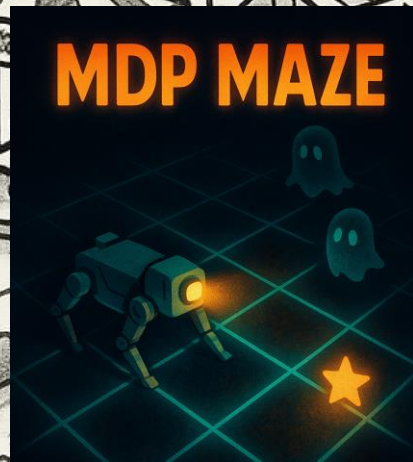
A type of AI that learns by trial and error to make better decisions.



ANYmal

- Autonomous robot for exploring difficult environments.
- Dynamic walking and climbing. Perception of environment through lasers and cameras.
- Ideal for inspection of industrial facilities. Several hours of autonomous walking. Light and easy to transport.
- <https://rsl.ethz.ch/robots-media/anymal.html>





How do we find the optimal strategy?

Find the action that yields the highest cumulative reward!

How do we compute the cumulative reward?

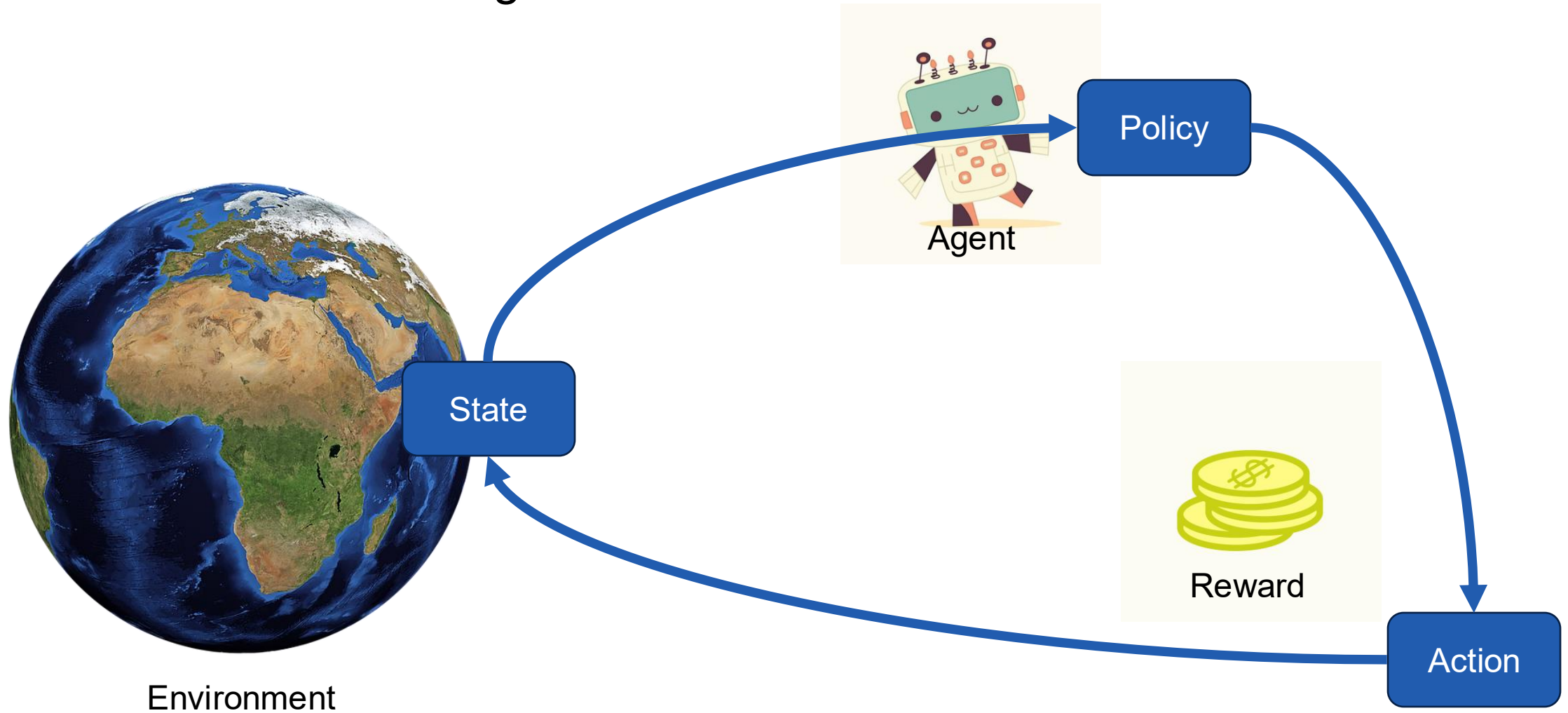
Use the Bellman equations!

How do we compute the Bellman equations?

Use Robbins-Monro

Let's play game 1

Reinforcement learning:



Flying to the moon

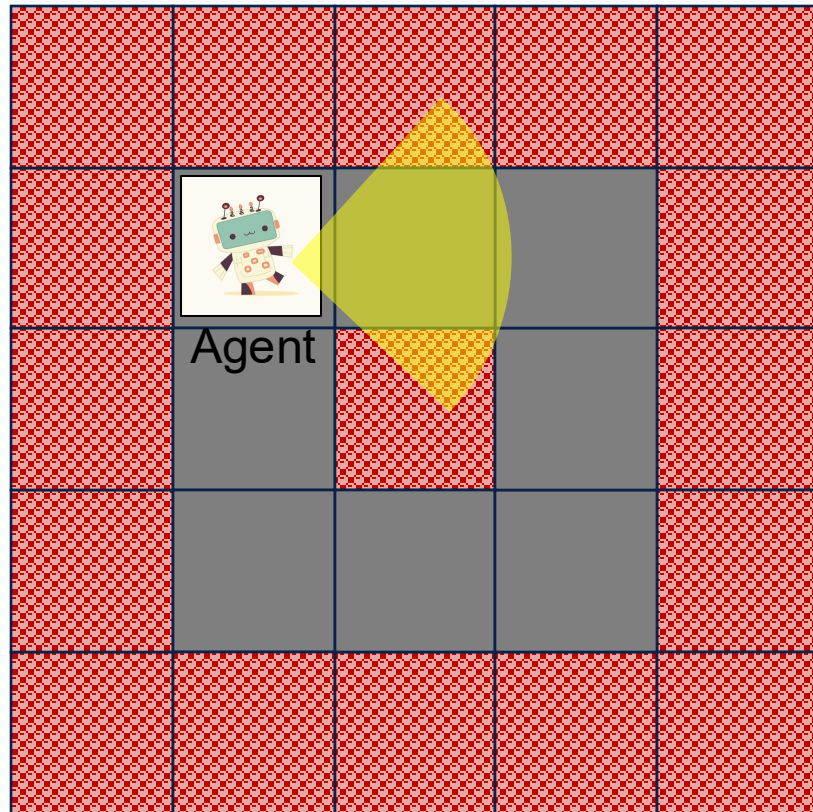


Investing in stock market



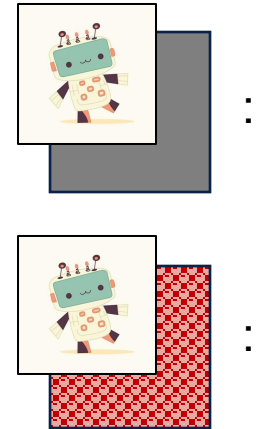
Example

Environment

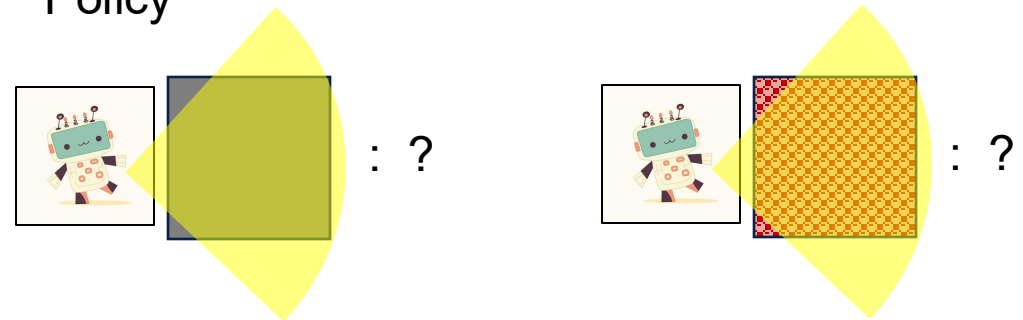


Actions

Rewards

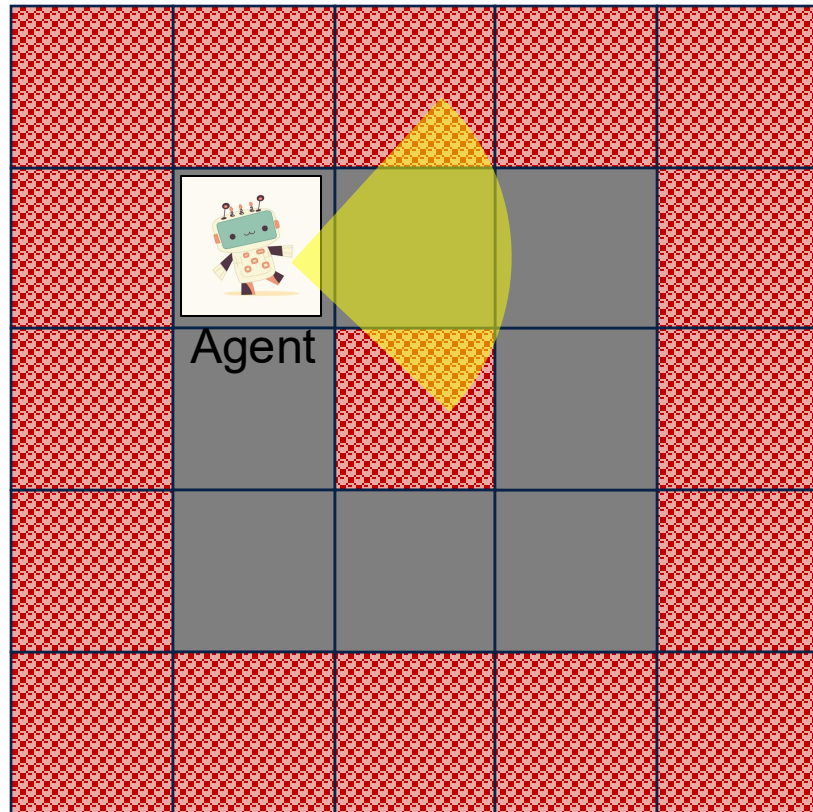


Policy

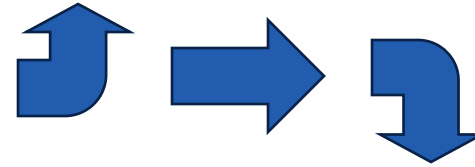


Example

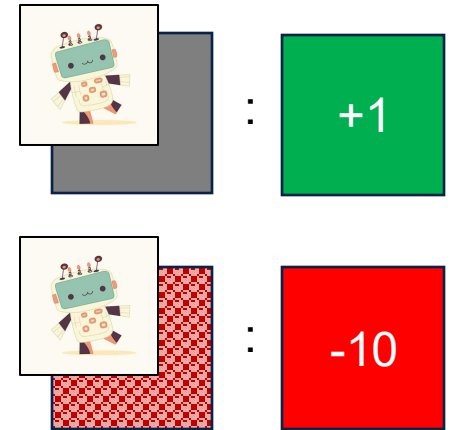
Environment



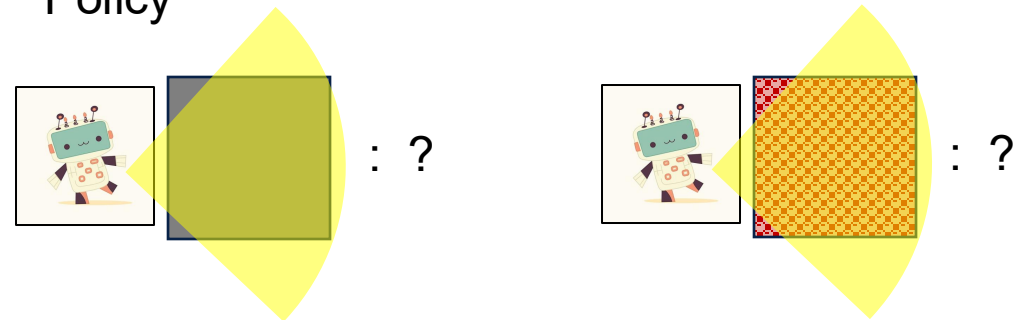
Actions



Rewards

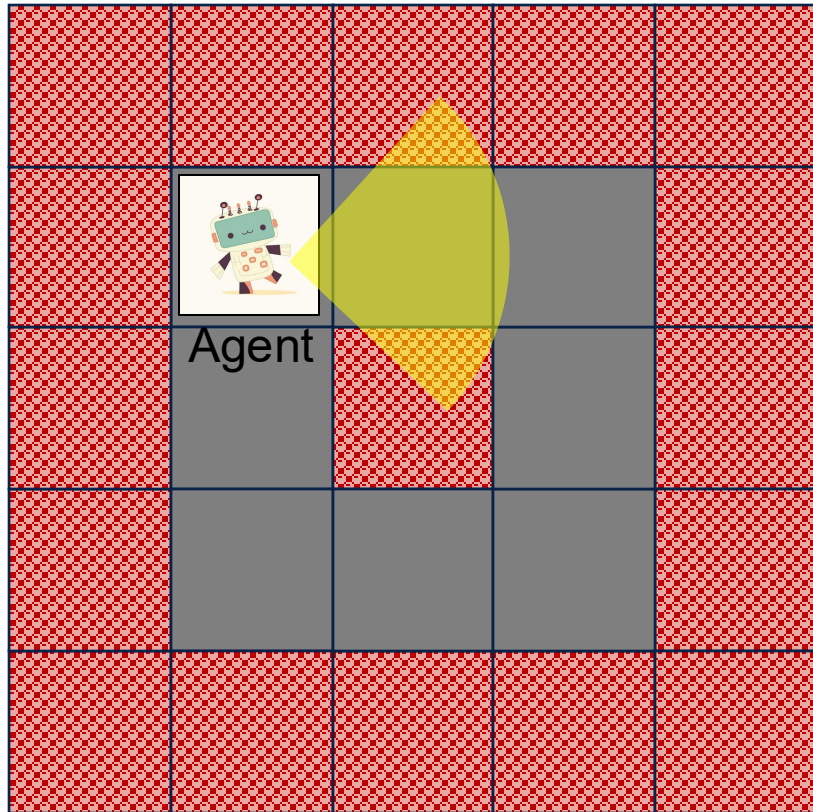


Policy

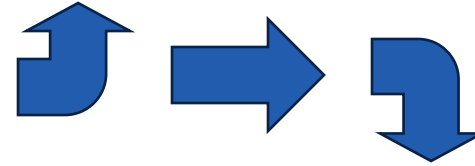


Example

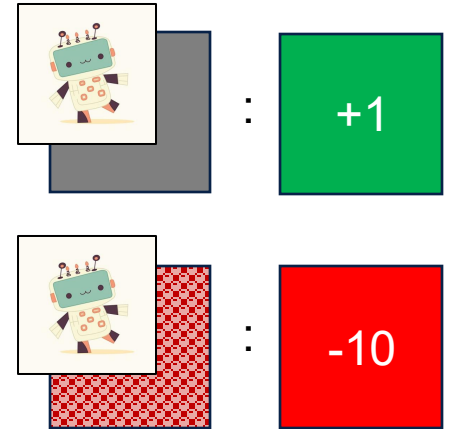
Environment



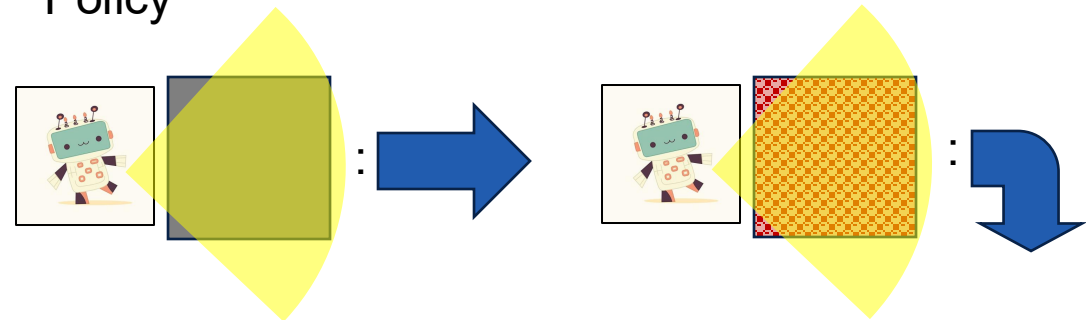
Actions



Rewards



Policy



Example

Environment



Set of $n \leq 12$ matches

Actions

Take 1, 2, or 3 matches.

Rewards

You take last match :

+1

Opponent takes last match :

-1

Policy

?

Example

Environment



Set of $n \leq 12$ matches

Actions

Take 1, 2, or 3 matches.

Rewards

You take last match :

+1

Opponent takes last match :

-1

Policy

Take $n \% 4$ matches, when $n \% 4 \neq 0$.

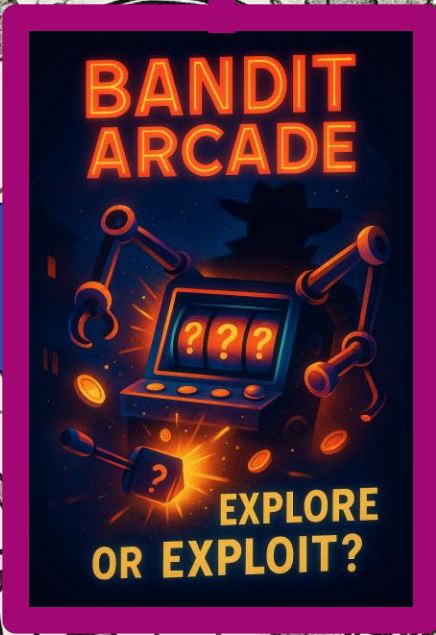
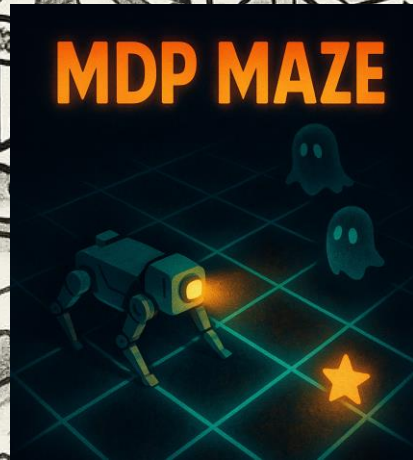
Markov-decision processes

Markov-decision processes

- A discrete *Markov decision process* is a tuple (S, A, P, R, γ) defined by the following:
 - S is a finite set of states.
 - A is a finite set of actions.
 - P describes a set of transition probabilities. For $s, s' \in S$ and $a \in A$, $P(s' \mid s, a)$ is the probability of ending in state s' after taking action a at state s .
 - ***P induces a transition function that from a state and an action produces a new state!***
 - R is a function assigning each triple $(s, a, s') \in S \times A \times S$ to a reward $R(s, a, s') \in \mathbb{R}$.
 - $\gamma \in (0, 1)$ is the discount factor. This can be ignored at the moment. It is a constant used to compute the value of a policy.

The goal of reinforcement learning

- Given an MDP, discover a policy (i.e. a strategy) that maximizes the cumulative reward!
- A policy is a function that decides from a given state which action to take.
- A policy induces a sequence of states, actions, and rewards.
- The goal is to discover the policy that maximizes the sum of all rewards.

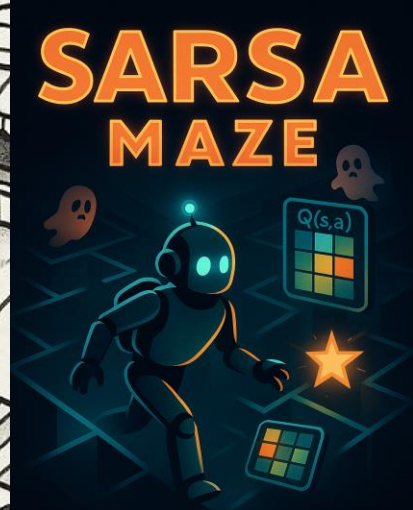


How do we find the optimal strategy?

Find the action that yields the highest cumulative reward!

How do we compute the cumulative reward?

Use the Bellman equations!



How do we compute the Bellman equations?

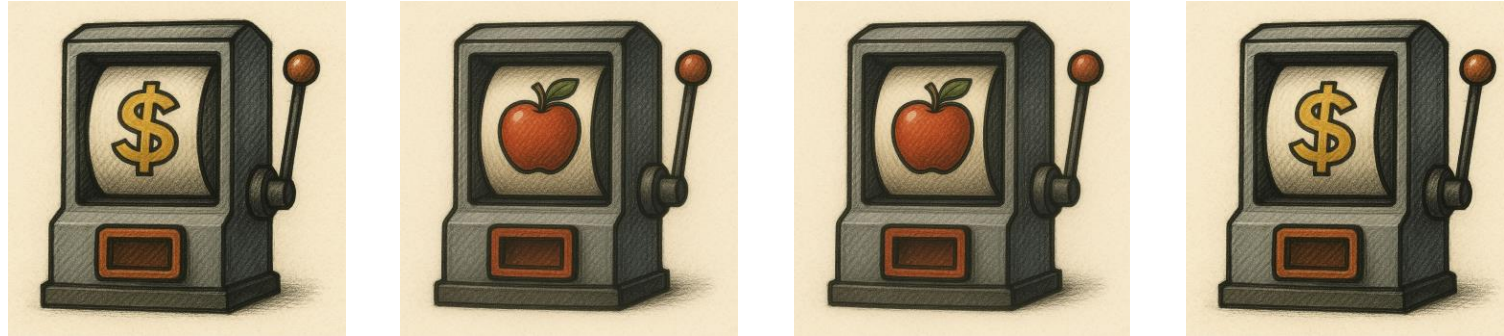
Use Robbins-Monro

Exploration and exploitation

Exploration vs exploitation

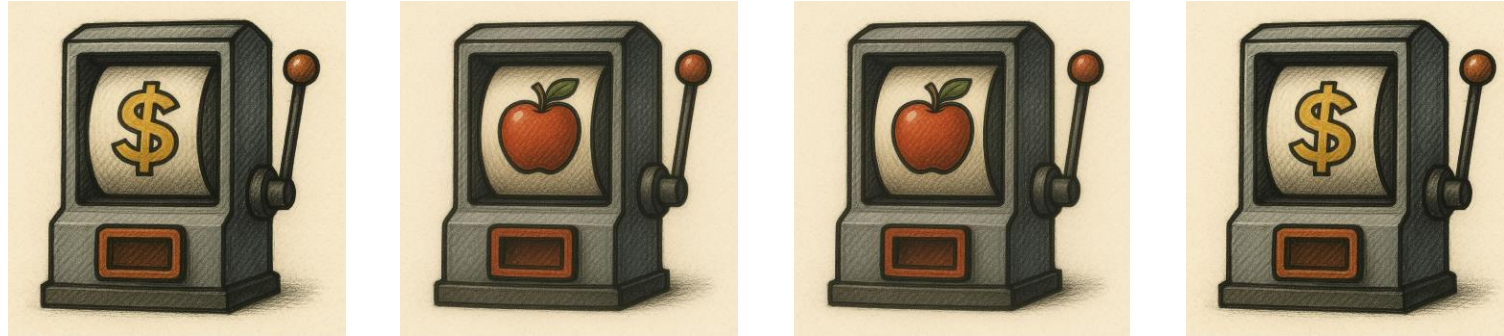
- When interacting with an MDP, one must achieve two goals:
 - **[Exploration]** Gather data to understand the transition probabilities and the rewards.
 - **[Exploitation]** Discover the optimal policy.
- These two are usually in conflict, so we must find a way to balance them.

Example: casino slot machines



- You are given 4 slot machines.
- Each of them can give you 1 CHF after pulling the lever if you get the \$ symbol.
- For each machine, the probability of getting \$ is different.
- **How do we maximize our reward over time?**

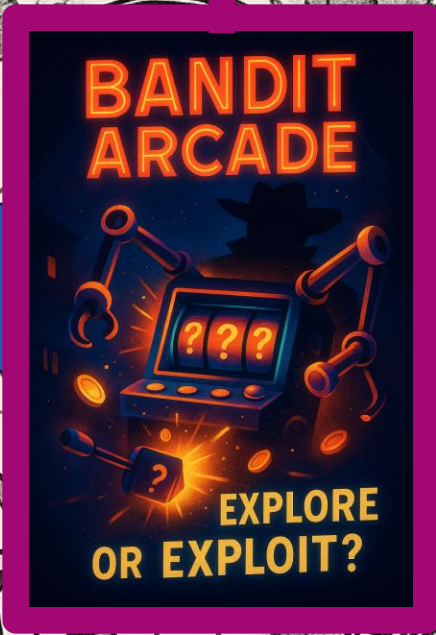
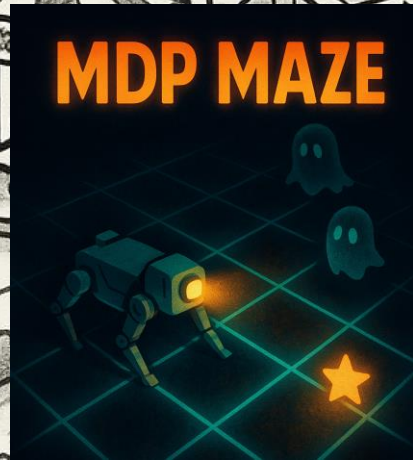
Example: casino slot machines



- Note that this is an MDP. There are states, actions, rewards, and transitions.
- **It is also an instance of bandits, just that the urns are now slot machines (bandits).**
- We already know UCB as a great algorithm for winning this game.
- In RL, however, UCB is an overkill and is very computationally consuming.

ϵ -greedy

- A simpler yet still powerful method to optimize this game is the ϵ -greedy algorithm.
- The strategy is similar to the urn games. For each slot machine m , we have an empirical estimator $\widehat{p}_m$.
- Before you play the game, you define a value $\epsilon \in [0,1]$ which quantifies the following strategy:
 - With probability ϵ , you pick a slot machine totally at random.
 - With probability $1 - \epsilon$, you pick the most promising slot machine.
- Before you play, you then create an urn whose probability of drawing white is ϵ .
- Then you start to play. At each round, you pick a slot machine as follows:
 - Draw a ball from the urn.
 - If the ball is white, then you pick a slot machine totally at random.
 - Otherwise, you pick the slot machine whose empirical estimator is the highest.
 - Depending on the reward you get from the slot machine, you update its corresponding empirical estimator.

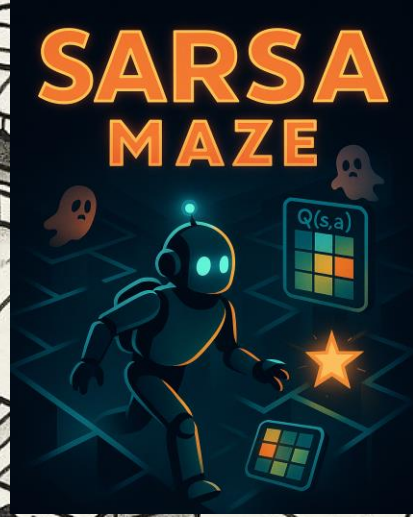


How do we find the optimal strategy?

Find the action that yields the highest cumulative reward!

How do we compute the cumulative reward?

Use the Bellman equations!

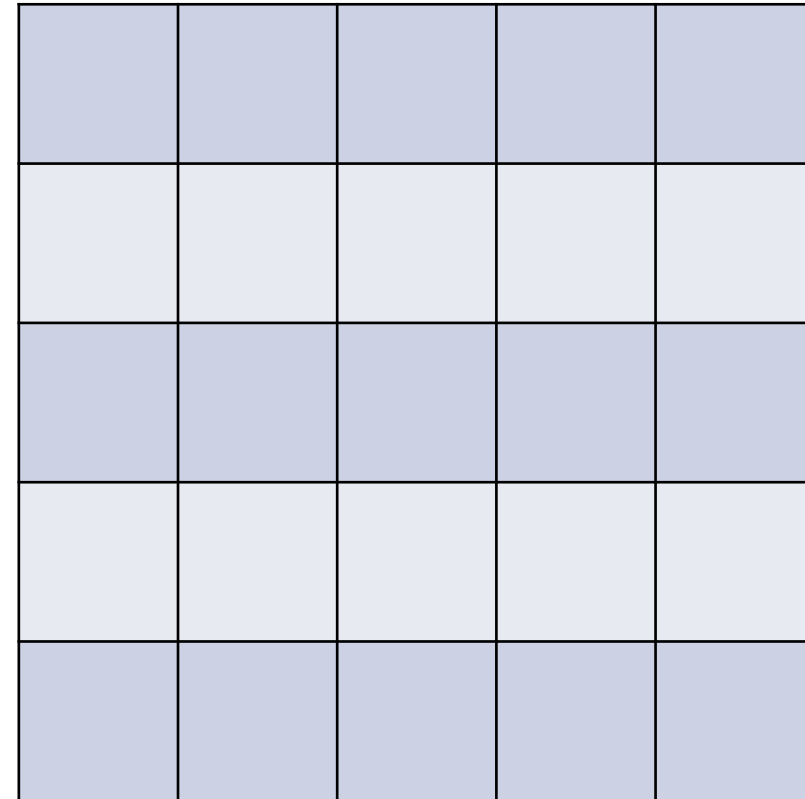
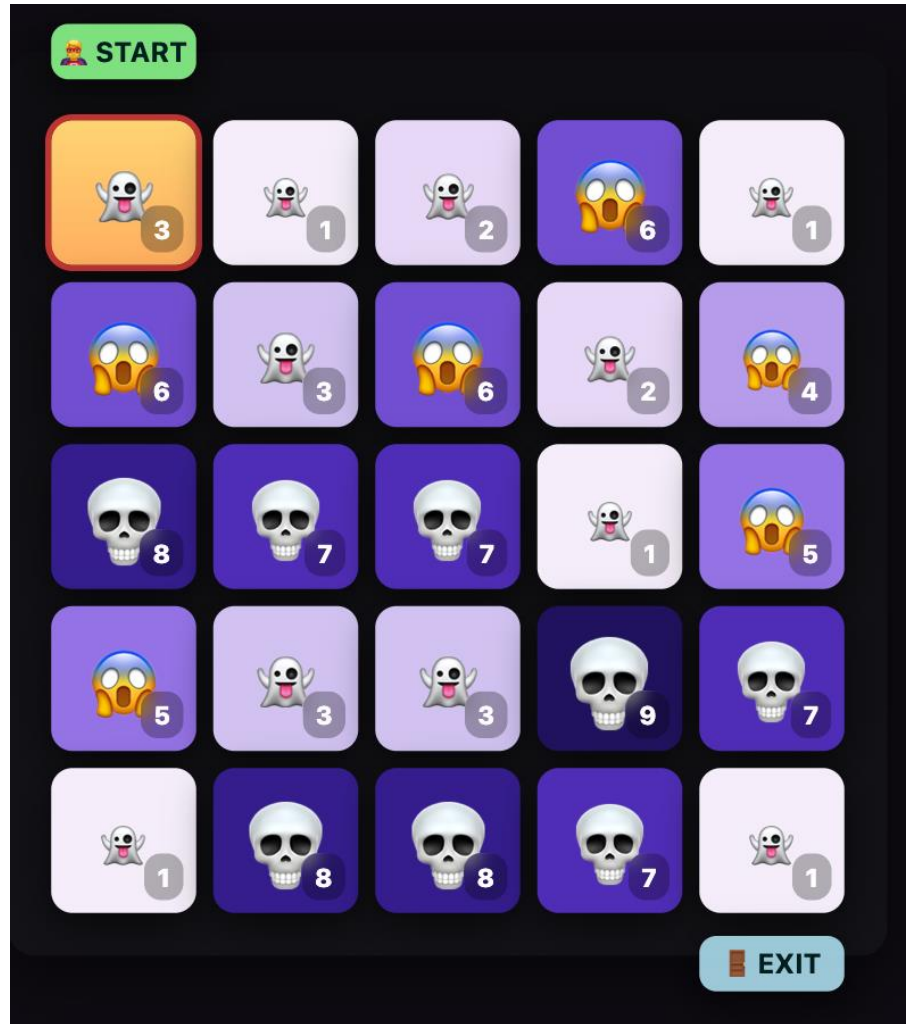


How do we compute the Bellman equations?

Use Robbins-Monro

Bellman equations

What is the maximum reward?



What did you discover?

Bellman equation

- Suppose the following:
 - You are at state s .
 - You take action a .
 - You reach state s' .
 - You then take action a' .
- Consider the following notation:
 - $Q(s, a)$, the total reward you can expect from s after taking action a .
 - $R(s, a, s')$, the immediate reward of taking action a at state s and reaching state s' .
- Then

$$Q(s, a) = R(s, a, s') + Q(s', a')$$

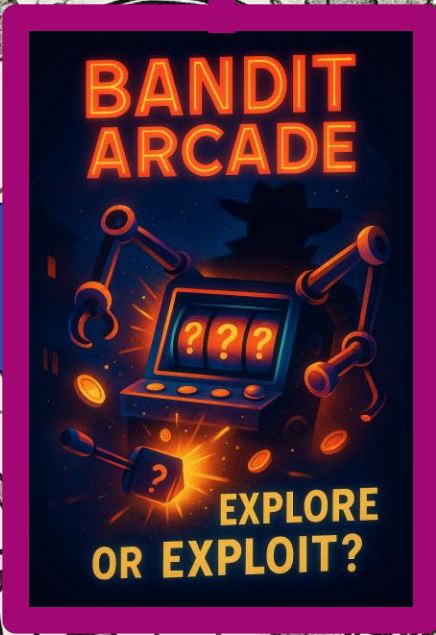
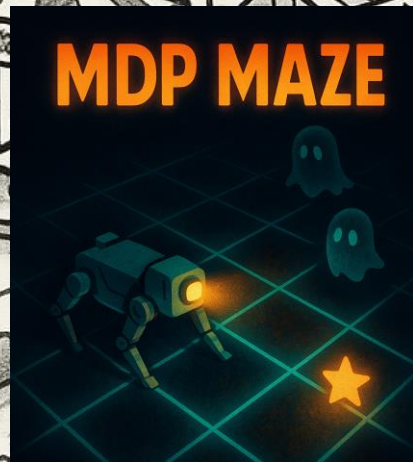
Bellman equation

- When the MDP can go forever, then we need to add a **discount factor** γ .

$$Q(s, a) = R(s, a, s') + \gamma Q(s', a').$$

- A stronger version is the following

$$Q(s, a) = R(s, a, s') + \gamma \max_{a' \in A} Q(s', a').$$

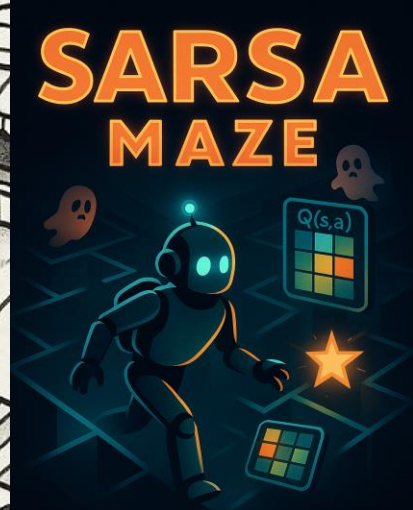


How do we find the optimal strategy?

Find the action that yields the highest cumulative reward!

How do we compute the cumulative reward?

Use the Bellman equations!



How do we compute the Bellman equations?

Use Robbins-Monro

Robbins Monro

Darts in the dark game!

Use the keyboard to play.
See "Key bindings" on the right.

90
50
10

Scoreboard

Last score:
— / 100

Last distance from bullseye
unknown

Average score
—

Best score
—

True bullseye position
hidden

Your position
65.0

Round 0

Round history

Shoot to start collecting data. Try to infer where the invisible bullseye is!

Key bindings:

- ↑ Move up
- ↓ Move down
- Space Shoot

Another game: the cubic root of 2025

Can you formalize the algorithm for optimizing this game?

The Robbins-Monro update rule

- Let θ_t be your estimate for the bullseye location at time t .
- Let α_t be a **positive** scaling factor, usually called **learning rate**. It decays with time.
- Let θ_t^* be the actual location of the bullseye.
- Then your estimators can be described as follows.

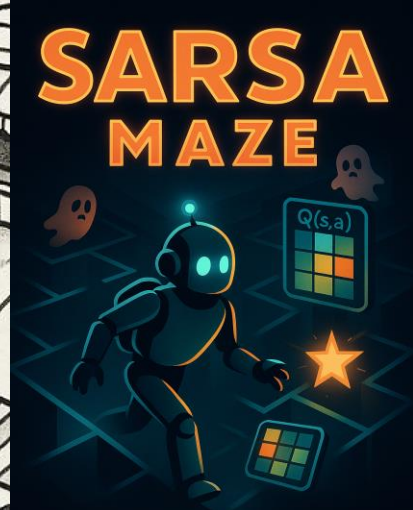
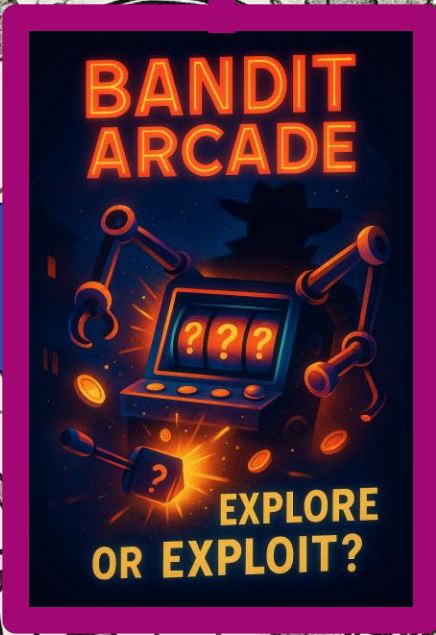
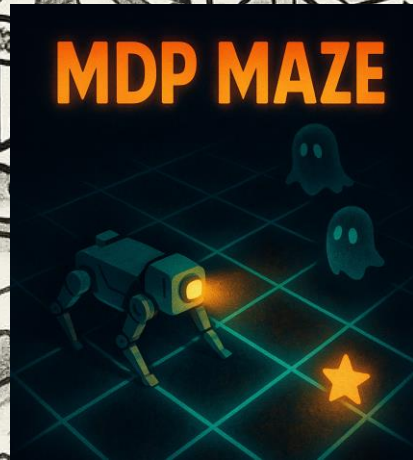
$$\theta_{t+1} \leftarrow \theta_t + \alpha_t (\theta_t^* - \theta_t).$$

What happens in the darkness?

- You don't see θ_t^* , but you can “measure” $\Delta_t = \theta_t^* - \theta_t$. The system reports it to you!
- The Robbins-Monro update rule happens to converge to the minimize the distance to the bull's eye in the asymptotic limit if $\alpha_t = \frac{1}{t}$.

Applications in practice

- In practice, we want to estimate a particular unknown value, like
 - the bullseye's location or
 - the value of a commodity that fluctuates over time or
 - the probability of getting money when we pull the lever of a slot machine.
 - **the value of $Q(s, a)$ after taking action a at a particular state s in an MDP.**
- After each guess, we get feedback from the environment on how well our estimate is.
- The Robbins-Monro algorithm gives us a way to estimate all these things!
- This algorithm has led to many interesting algorithms in ML, like SARSA and stochastic gradient descent!



How do we find the optimal strategy?

Find the action that yields the highest cumulative reward!

How do we compute the cumulative reward?

Use the Bellman equations!

How do we compute the Bellman equations?

Use Robbins-Monro

SARSA

Blackjack

Current Sum
14

Episode Reward
14

Cards Drawn
3

Games Played
0

Total Reward
14

1 4 9

HIT STICK NEW GAME

✓ Drew 9! Sum = 14. Reward = +9. Episode Reward = 14

Blackjack

Game Rules

- You start with a sum of **0**
- Cards are random integers from **1 to 10**
- **HIT:** Draw a card and add it to your sum
 - If $\text{sum} \leq 21$: Get **reward = card value**
 - If $\text{sum} > 21$: **BUST!** Game ends with **reward = -sum**
- **STICK:** End the game with **reward = 0**
- **Goal:** Maximize your total reward across all games!

What is the optimal policy?

- The optimal policy must have the following form:
- Sum = 0 -> HIT
- Sum = 1 -> HIT
- Sum = 2 -> HIT
- ...
- Sum = 20 -> STICK
- Sum = 21 -> STICK

- **When do we switch from HIT to STICK?**

What is the optimal policy? The answer is in the Bellman equation!

$$Q(s, a) = R(s, a, s') + \gamma Q(s', a').$$

- The only problem here is that this time we do not have $Q(s, a)$.
- In practice, this is the main issue: $Q(s, a)$ is unknown.
- So we must estimate it.
- Once known, the optimal policy is the following: at state s , pick the action a that yields the largest $Q(s, a)$.

Estimating Q

$$Q(s, a) = R(s, a, s') + \gamma Q(s', a').$$

- We start by creating a table $\hat{Q}$ where $\hat{Q}(s, a)$ is initially random, for each possible state s and action a .
- We then **explore** the MDP by picking actions at random. Through this exploration, we update $\hat{Q}$ as follows. Suppose that we take a at s , reaching s' , collecting r , and then taking a' . We wish to have

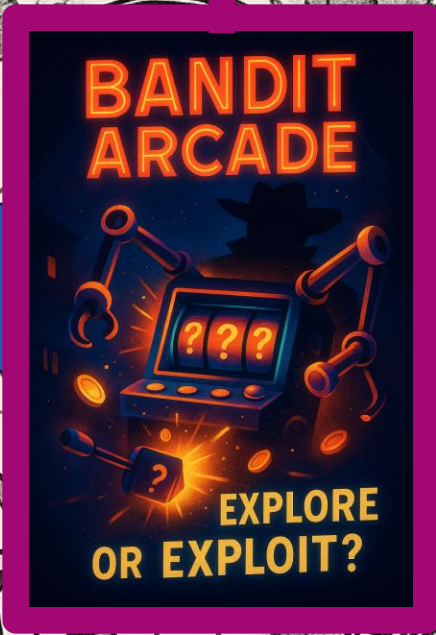
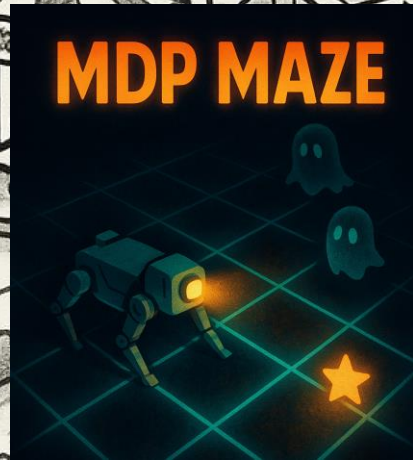
$$\hat{Q}(s, a) = r + \gamma \hat{Q}(s', a').$$

- Let's call $r + \gamma \hat{Q}(s', a')$ the "bullseye" 😊 Two things may happen, either $\hat{Q}(s, a)$ is above or below the bullseye. We must then adjust $\hat{Q}(s, a)$ as we did in the Robbins-Monro algorithm.

$$\hat{Q}(s, a) \leftarrow \hat{Q}(s, a) + \alpha (\text{bullseye} - \hat{Q}(s, a)).$$

SARSA: an algorithm to find Q

- We have then conceived an algorithm to estimate Q , which leads us to a policy to maximize rewards 😊
- We play with the MDP by **exploring** actions at random and then updating an estimator $\hat{Q}$ using the Robbins-Monro formula.
- But this is not enough! Note that **exploration** is not the same as **exploitation**. At some point, we need to start playing the MDP not in an exploratory way, but in an exploitative way, so we can get rewards!
- How do we do this? We use the ϵ -greedy strategy at state s
 - Sometimes we take a random action to **explore**.
 - Sometimes we take the action a that maximizes $\hat{Q}(s, a)$ (**exploit**).
- This algorithm is called SARSA.

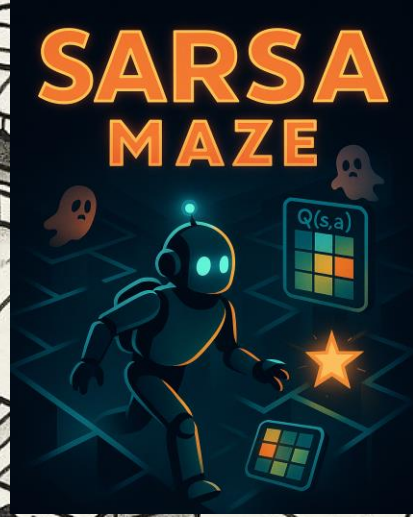


How do we find the optimal strategy?

Find the action that yields the highest cumulative reward!

How do we compute the cumulative reward?

Use the Bellman equations!



How do we compute the Bellman equations?

Use Robbins-Monro

Applications

HDPlanner

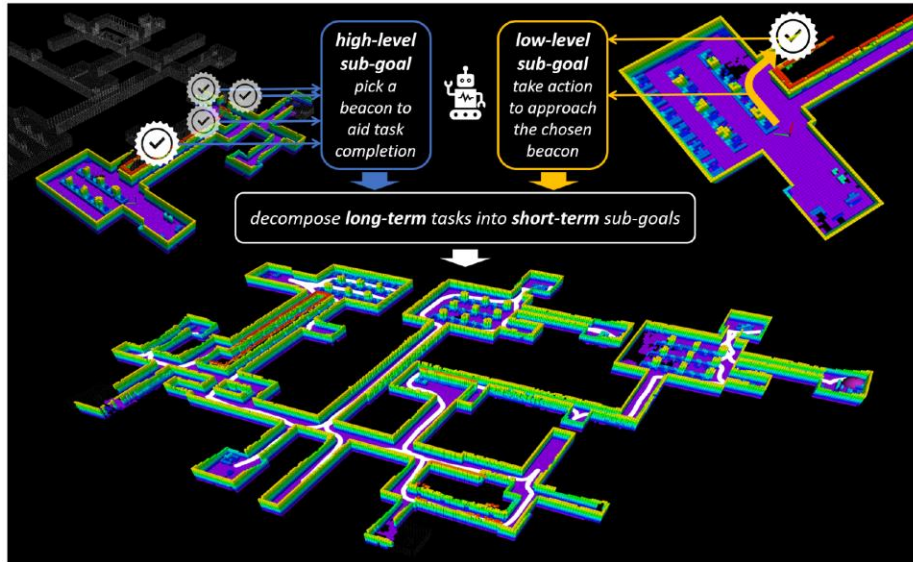


Image from Liang et al 2024, **HDPlanner: Advancing Autonomous Deployments in Unknown Environments through Hierarchical Decision Networks**

- **Problem Statement:**
 - Robots face challenges in **exploring unknown areas** or **navigating to targets** without prior maps.
 - Real-time decision-making is needed in **dynamic, uncertain environments**.
- **Key Goals:**
 - Minimize travel distance and time.
 - Balance exploration (gathering new information) and exploitation (using current knowledge).
- **Why It's Hard:**
 - Environments are **partially observable**, requiring adaptive and hierarchical reasoning.

HDPlanner

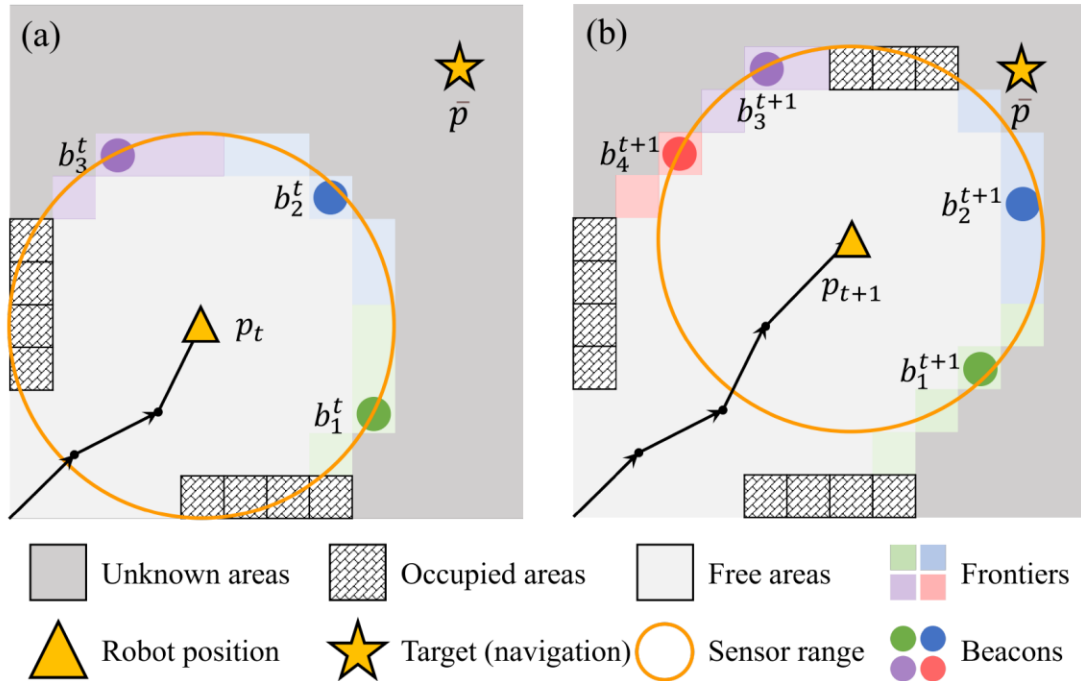


Image from Liang et al 2024, **HDPlanner: Advancing Autonomous Deployments in Unknown Environments through Hierarchical Decision Networks**

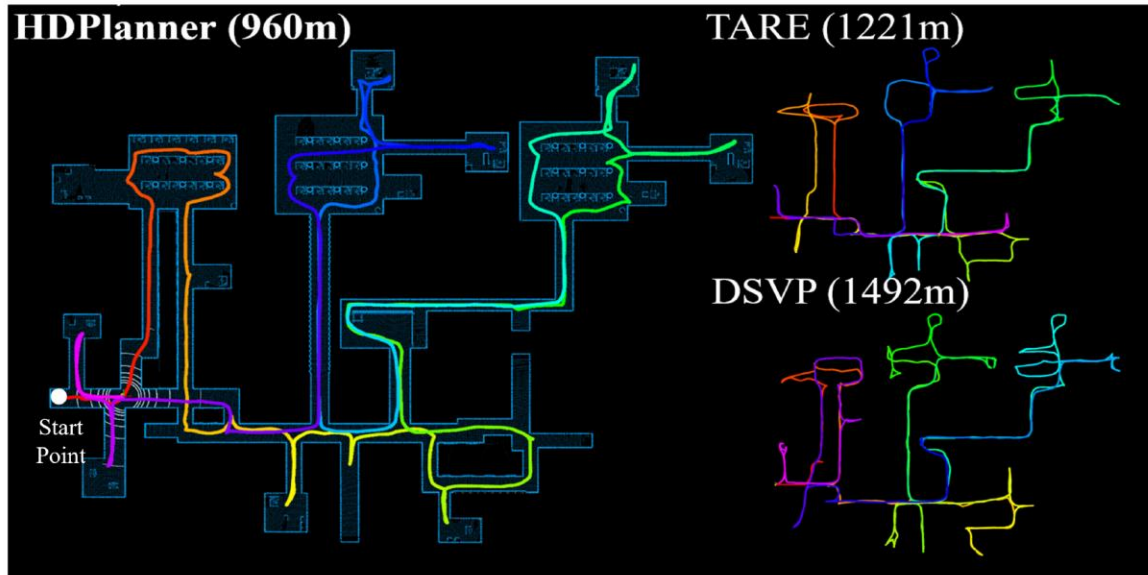
- **Approach Overview:**

- Hierarchical RL-based framework for autonomous **exploration** and **navigation**.
- Decomposes complex tasks into:
 - **High-level decisions:** Select "beacons" (regions of interest).
 - **Low-level decisions:** Plan waypoints to move towards beacons.

- **Technical Highlights:**

- Uses **hierarchical attention networks** for better reasoning across scales.
- Employs **contrastive learning** to improve decision-making robustness.

HDPlanner



- **Impact:** Outperformed benchmarks by up to:
 - **35.7% reduction** in travel distance for exploration.
 - **16.5% improvement** in navigation efficiency.

Image from Liang et al 2024, **HDPlanner: Advancing Autonomous Deployments in Unknown Environments through Hierarchical Decision Networks**

Why this matters



- **Value Proposition:**
 - Enables **efficient operations** in logistics, warehouse automation, and autonomous vehicles.
 - Reduces operational costs with **shorter paths** and **faster navigation**.
- **Real-World Validation:**
 - Tested in **simulations** (300 maps) and **real-world environments** (indoor and outdoor).
 - Demonstrates **robust adaptability** to complex, dynamic settings.
- **Key Takeaway:**
 - RL-driven frameworks like HDPlanner can provide scalable solutions to real-world challenges in autonomy.

Reinforcement learning for finance

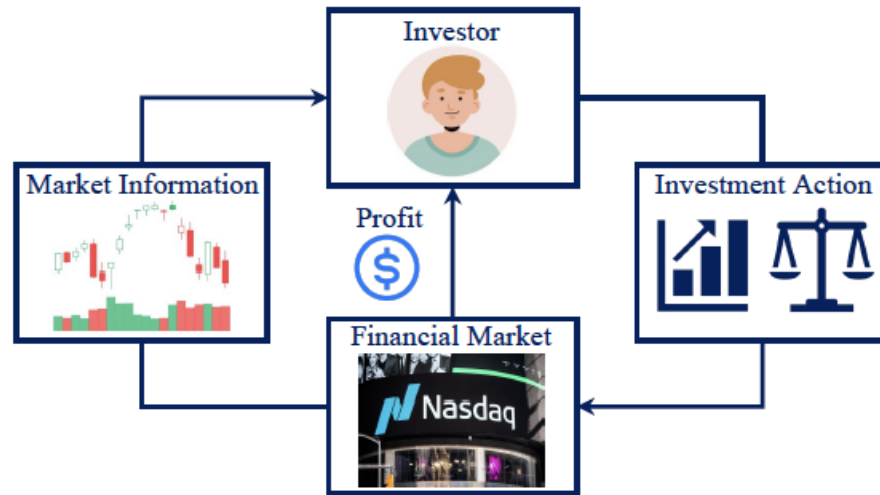


Image from Sun et al 2024 TradeMaster: A Holistic Quantitative Trading Platform Empowered by Reinforcement Learning.

- **Applications**

1. **Algorithmic Trading**: Automating buy/sell decisions.
2. **Portfolio Optimization**: Allocating investments to maximize returns and minimize risks.
3. **Market Making**: Ensuring liquidity in financial markets.

- **Key Benefits**

- Data-driven decision-making.
- Potential to uncover profitable strategies (?)

Reinforcement learning for finance



- **Key Challenges**

- 1. Noisy and Unpredictable Data:**

- Financial data has a low signal-to-noise ratio.
 - Market events are often unpredictable (e.g., black swan events).

- 1. Overfitting to Historical Data:**

- RL models risk being overly tuned to past trends and failing in live trading.

State-of-the-Art Limitations

- Limited reproducibility of results.
- High dependency on simulated environments that may not reflect real-world markets.

Practical concerns in RL



- RL usually requires large amounts of example data.
- Lack of interpretability.
- Requires expensive computational resources to be trained.
- Ethical dilemmas.